Array Search Algorithms

Objective: To introduce fundamental algorithms for finding specific elements within arrays and understand their efficiency.

Linear Search

Linear search (also known as sequential search) is the most straightforward search algorithm. It checks each element in the array one by one, from the beginning, until the target element is found or the end of the array is reached.

Think of looking for a specific item in a grocery list without any order. You would read each item from top to bottom until you find the one you're looking for.

Code Solution (Python)

```
def linear_search(arr, target):
    """
    Searches for a target element in an array using linear search.

    Args:
      arr: The array to search in.
      target: The element to search for.

    Returns:
      The index of the target element if found, otherwise -1.
    """
    for i in range(len(arr)):
        if arr[i] == target:
            return i  # Target found at index i
    return -1  # Target not found
```

Big-O Notation

Worst-Case Scenario: The target element is the very last element in the array, or it is not present in the array at all. In this case, the algorithm must examine every single element.

Time Complexity: O(n)

Average-Case Scenario: The target element is, on average, somewhere in the middle of the array. The algorithm will examine roughly half of the elements.

Time Complexity: O(n) (Constant factors are dropped in Big-O)

Best-Case Scenario: The target element is the first element in the array. The algorithm finds it immediately in the first comparison.

Time Complexity: O(1)
Space Complexity: O(1) because it only uses a few variables to keep track of the index and the target, regardless of the array size.

When to Use Linear Search
When the array is unsorted.
When the array is relatively small.
When simplicity of implementation is the highest priority.

Binary Search
Binary search is a significantly more efficient search algorithm than linear search, but it has a crucial requirement: the array must be sorted. It works by repeatedly dividing the search interval in half.

Here's the process:

Find the middle element of the current search interval.
Compare the middle element with the target:
If they match, the element is found.
If the target is less than the middle element, discard the right half of the interval (including the middle element) and search in the left half.
If the target is greater than the middle element, discard the left half of the interval (including the middle element) and search in the right half.
Repeat the process on the remaining half until the target is found or the interval becomes empty.
Think of finding a word in a dictionary. You don't start from the beginning; you open it somewhere in the middle, and depending on whether the word you're looking for comes before or after the currently opened page, you refine your search to either the first or second half of the remaining pages.

Code Solution (Python)

```
def binary_search(arr, target):
    """
    Searches for a target element in a sorted array using binary search.

    Args:
      arr: The sorted array to search in.
      target: The element to search for.
```

```
    Returns:
      The index of the target element if found, otherwise -1.
    """
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2  # Calculate the middle index

        # Check if target is present at mid
        if arr[mid] == target:
            return mid

        # If target is greater than the middle element, search in the right half
        elif target > arr[mid]:
            low = mid + 1

        # If target is less than the middle element, search in the left half
        else:
            high = mid - 1

    # If the target is not found
    return -1
```

Big-O Notation

The efficiency of binary search comes from repeatedly halving the search space.

Worst-Case Scenario: The target is not in the array, or it's at one of the ends, requiring the algorithm to continue dividing the array until the search space is empty.
Time Complexity: O(logn)
Average-Case Scenario: Similar to the worst-case, the search space is repeatedly halved.
Time Complexity: O(logn)
Best-Case Scenario: The target element is the middle element in the initial step.
Time Complexity: O(1)
Space Complexity: O(1) as it only uses a few variables (low, high, mid).

When to Use Binary Search
When the array is sorted.
When efficiency is important, especially for large arrays.
Interpolation Search
Interpolation search is an improvement over binary search, particularly useful when the elements in the sorted array are uniformly distributed (e.g., numbers in sequence, evenly spread values). Instead of always checking the middle element, it estimates a more likely position for the target based on its value relative to the values at the beginning and end of the current search interval.

Think of searching for a specific year in a sorted list of historical dates. If you're looking for a date closer to the modern era, you'd guess a position further down the list rather than exactly the middle. Interpolation search uses this intuition.

The formula for estimating the position (pos) is:

$$pos = low + ((target - arr[low]) * (high - low)) / (arr[high] - arr[low])$$

Where:

low: the index of the lower bound of the search interval.
high: the index of the upper bound of the search interval.
target: the element being searched for.
arr: the sorted array.

Code Solution (Python)

```
def interpolation_search(arr, target):
    """
    Searches for a target element in a sorted, uniformly distributed array
    using interpolation search.

    Args:
      arr: The sorted array to search in.
      target: The element to search for.

    Returns:
      The index of the target element if found, otherwise -1.
    """
    low = 0
```

```
    high = len(arr) - 1

    while low <= high and target >= arr[low] and target <= arr[high]:
        # Avoid division by zero and issues with non-uniform data
        if arr[high] == arr[low]:
            if arr[low] == target:
                return low
            else:
                return -1

        # Estimate the position using the interpolation formula
        # Use integer division //
        pos = low + (
            (target - arr[low]) * (high - low)
        ) // (arr[high] - arr[low])

        if arr[pos] == target:
            return pos  # Target found
        elif target > arr[pos]:
            low = pos + 1  # Search in the upper part
        else:
            high = pos - 1  # Search in the lower part

    return -1  # Target not found
```

Big-O Notation

The performance of interpolation search is highly dependent on the distribution of values in the array.

Worst-Case Scenario: The data is not uniformly distributed (e.g., values are clustered at one end). In this case, the algorithm might still check most of the elements.
Time Complexity: O(n)
Average-Case Scenario: The elements are uniformly distributed. Interpolation search typically performs better than binary search in this case.
Time Complexity: O(loglogn) (This grows much slower than O(logn))
Best-Case Scenario: The target element is found in the very first interpolation step.
Time Complexity: O(1)
Space Complexity: O(1), as it uses a constant amount of extra space.

When to Use Interpolation Search

When the array is sorted.
When the elements are uniformly distributed.
When you need potentially faster performance than binary search on specific datasets.